

SECS/GEM Machine Simulator and Middleware Assessment

Documentation & Report

Candidate	Andre Murojisu
Assessment	SECS/GEM Simulator and Middleware
Document Version	1.0
Date	26 August 2026

This report documents the implemented prototype, simulator configuration, middleware architecture, communication flow, telemetry mapping, setup procedure, and known limitations.

1. Solution Overview

The solution consists of two independent machine simulators and one middleware application. Each simulator exposes a TCP listener and exchanges SECS/GEM-style messages with the middleware. The middleware manages both machine connections in parallel, performs the required setup sequence, receives equipment-generated S6F11 events, returns S6F12 acknowledgements, maps selected events to telemetry payloads, and sends the payloads to an HTTP endpoint.

Component	Address	Purpose
Machine Simulator A	127.0.0.1:5001	Manual-oriented equipment simulation (MG22)
Machine Simulator B	127.0.0.1:5002	Generic SECS/GEM-style equipment simulation
Middleware	Client to ports 5001 and 5002	Parallel communication, event processing, telemetry transmission
HTTP endpoint	Configured locally	Receives JSON telemetry by HTTP POST

The implementation is intentionally scoped as an assessment prototype. It demonstrates message sequencing, report/event configuration, asynchronous event handling, and telemetry delivery without claiming to be a production HSMS/SECS-II stack.

1.1 Key Implementation Characteristics

- Machine A and Machine B are handled by separate middleware threads so the two connections operate independently.
- Machine A uses the supplied equipment-specific identifiers and setup sequence, including REMOTE and PPSELECT.
- Machine B uses symbolic generic identifiers such as RPTID1, SVID1, and CEID1 for the simulated scope.
- S6F11 events are acknowledged with S6F12 before telemetry is sent by HTTP POST.
- The HTTP endpoint is intentionally left empty in source control and configured locally when testing.

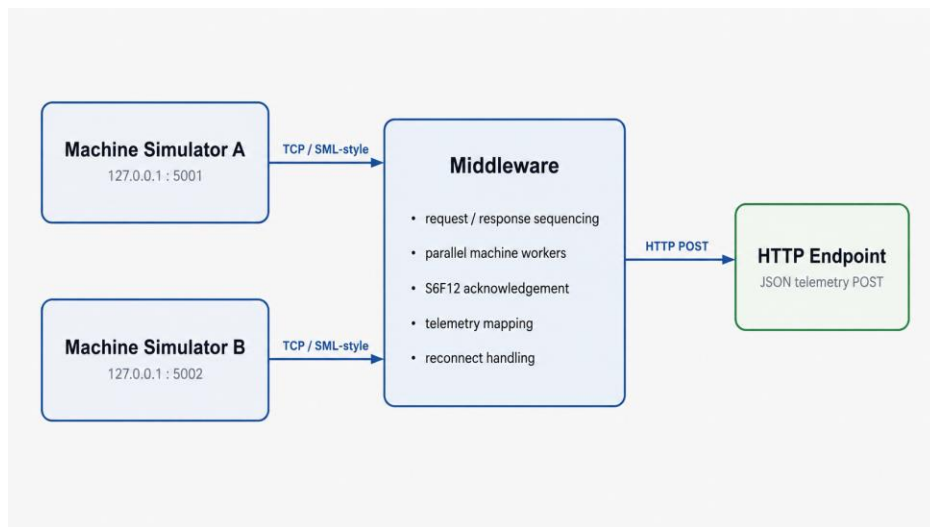


Figure 1. High-level solution architecture.

2. Machine Simulator A

Machine Simulator A models an MG22 equipment interface on 127.0.0.1:5001. Its setup sequence follows the supplied communication examples and preserves the numeric report, variable, event, and status identifiers used by that equipment scope.

2.1 Configuration

Item	Value
Equipment model	MG22
Software revision	3.7.0.0
Listen address	127.0.0.1
Listen port	5001
Initial simplified control state	1
Online Remote control state	5

2.2 Report Definitions

RPTID	Variables	Description
1	1835, 1836	jobIdLastStartedLot; lotIdLastStartedLot
2	1830, 1831	jobIdLastFinishedLot; lotIdLastFinishedLot
3	4306	mapResultLastMap

2.3 Event-to-Report Links

CEID	Linked Report
4	RPTID 1
5	RPTID 2
140	RPTID 3
141	RPTID 3

The enable step also enables CEIDs 13, 130, and 131 in addition to the report-linked events. CEID 13 (processRecipeSelected) is used by the implemented telemetry demonstration and is emitted after PPSELECT.

2.4 Main Setup Sequence

- S1F13 / S1F14 - establish communications.
- S2F33 / S2F34 - clear existing reports, then define RPTID 1-3.
- S2F35 / S2F36 - clear existing event links, then link CEIDs to reports.
- S2F37 / S2F38 - disable all events, then enable the required CEIDs.
- S1F3 / S1F4 - query enabled events (SVID 12).
- S2F41 / S2F42 - request REMOTE control state.
- S1F3 / S1F4 - verify ControlState using SVID 11.
- S2F41 / S2F42 - send PPSELECT parameters.

- S6F11 / S6F12 - equipment event and host acknowledgement.



Figure 2. Machine A communication sequence implemented in the prototype.

3. Machine Simulator B

Machine Simulator B listens on 127.0.0.1:5002 and mirrors the same general report/event setup pattern as Machine A without copying Machine A-specific numeric identifiers. Symbolic identifiers are used to keep the simulation generic and explicit.

3.1 Generic Report and Event Mapping

Report		Variables
RPTID1		SVID1, SVID2
RPTID2		SVID3
Event	Linked Report	Application Name
CEID1	RPTID1	EVENT1
CEID2	RPTID2	EVENT2

3.2 Setup Sequence

Machine B keeps S2F33, S2F35, S2F37, and S1F3/S1F4 so that the simulator retains a realistic report/event configuration structure. Unlike Machine A, it does not execute S2F41 REMOTE or PPSELECT.

```
S2F33 W
<L[2]
  <U4[1] 1>
  <L[2]
    <L[2]
      <A[6] 'RPTID1'>
      <L[2]
        <A[5] 'SVID1'>
        <A[5] 'SVID2'>
      >
    >
  >
  <L[2]
    <A[6] 'RPTID2'>
    <L[1]
      <A[5] 'SVID3'>
    >
  >
>
```

S2F35 links CEID1 to RPTID1 and CEID2 to RPTID2. S2F37 then enables both events. A final S1F3 query requests SVID1, SVID2, and SVID3 and receives VALUE1, VALUE2, and VALUE3 in S1F4.

3.3 Event Generation

After the S1F3/S1F4 transaction is completed, Machine B triggers EVENT1 once. Delaying S6F11 until after S1F4 preserves the request/response ordering and prevents the unsolicited event from being mistaken for the S1F4 reply.

```
S6F11 W
<L[3]
  <U4[1] 1>
  <A[5] 'CEID1'>
  <L[1]
    <L[2]
      <A[6] 'RPTID1'>
      <L[2]
        <A[6] 'VALUE1'>
        <A[6] 'VALUE2'>
      >
    >
  >
>
```

Machine B Communication Flow

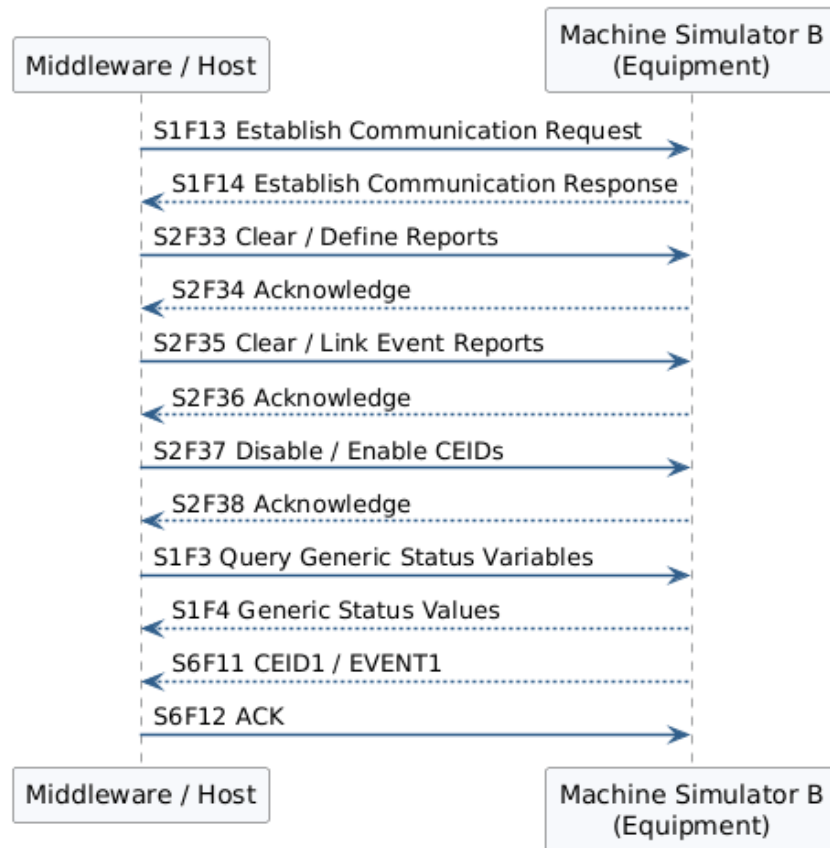


Figure 3. Machine B generic communication sequence.

4. Middleware Architecture and Implementation

The middleware is the host-side integration layer. It creates independent worker threads for Machine A and Machine B, so a connection problem or blocking receive on one machine does not prevent the other worker from running.

4.1 Parallel Machine Workers

```
Thread machineAThread = new Thread(RunMachineA);
Thread machineBThread = new Thread(RunMachineB);

machineAThread.Start();
machineBThread.Start();

machineAThread.Join();
machineBThread.Join();
```

Each worker maintains its own TcpClient, NetworkStream, StreamReader, and StreamWriter. The worker runs the corresponding setup sequence and then enters a passive receive loop for unsolicited equipment messages.

4.2 Connection Recovery

Both machine workers run inside a reconnect loop. Socket or I/O failures close the current client, wait for the configured reconnect delay (3000 ms), and retry the connection. This keeps the prototype usable when simulators are started or restarted independently.

4.3 Request/Response Validation

For setup messages, SendAndReceiveA and SendAndReceiveB send one request, read one framed response, and validate the response name (for example S2F34 or S1F4). An empty response or unexpected message produces an exception and returns control to the reconnect path.

4.4 Passive S6F11 Processing

```
if (message.StartsWith("S6F11")) {
    ProcessMachineBS6F11(writer, message);
    continue;
}
```

When S6F11 is received, the middleware immediately returns S6F12 with ACK code 00. The event is then mapped to an application-level event name and passed to the telemetry function.

5. SECS/GEM Communication Flow

The following table summarizes the message pairs implemented by the prototype. Message bodies are represented in SML-like text for readability.

Message	Direction	Purpose
S1F13 / S1F14	Host → Equipment	Establish communications
S2F33 / S2F34	Host → Equipment	Define or clear reports
S2F35 / S2F36	Host → Equipment	Link or clear event reports
S2F37 / S2F38	Host → Equipment	Enable or disable collection events
S1F3 / S1F4	Host → Equipment	Query status variables
S2F41 / S2F42	Host → Equipment	Machine A remote command / PPSELECT
S6F11 / S6F12	Equipment → Host	Event report and acknowledgement

5.1 Machine A Event Flow

Machine A accepts PPSELECT, emits CEID 13 (processRecipeSelected), and sends S6F11 with an empty report list for that event. The middleware replies with S6F12 and maps CEID 13 to the telemetry event name processRecipeSelected.

5.2 Machine B Event Flow

Machine B emits CEID1 / EVENT1 after its status-variable query completes. The event carries RPTID1 data values, receives S6F12 from the middleware, and is mapped to EVENT1 for telemetry transmission.

Telemetry Event Processing



Machine A: CEID 13 → processRecipeSelected | Machine B: CEID1 → EVENT1

Figure 4. Event-to-telemetry processing flow.

6. Telemetry Data Mapping and Transmission

The assessment middleware converts selected equipment events into compact JSON payloads. The endpoint is not committed in source code; it is configured locally when the solution is executed.

6.1 Machine A Mapping

Source	Mapped Value
Machine	MachineA
CEID	13
Event	processRecipeSelected
Timestamp	Local ISO-8601 timestamp with offset

```
{
  "machine": "MachineA",
  "timestamp": "2026-08-26T13:43:54+07:00",
  "ceid": 13,
  "event": "processRecipeSelected"
}
```

6.2 Machine B Mapping

Source	Mapped Value
Machine	MachineB
CEID	CEID1
Event	EVENT1
Timestamp	Local ISO-8601 timestamp with offset

```
{
  "machine": "MachineB",
  "timestamp": "2026-08-26T13:44:04+07:00",
  "ceid": "CEID1",
  "event": "EVENT1"
}
```

6.3 HTTP Transmission

The middleware serializes the payload as UTF-8 JSON and sends it using HTTP POST with Content-Type and Accept set to application/json. The request timeout and read/write timeout are both 10 seconds. HTTP and application errors are logged without terminating the other machine worker.

```
HttpRequest request = (HttpRequest)WebRequest.Create(url);
request.Method = "POST";
request.ContentType = "application/json";
request.Accept = "application/json";
request.ContentLength = data.Length;
```

7. Verification Results

End-to-end testing confirmed that both machine event paths reached the HTTP endpoint successfully. The endpoint returned an accepted response containing the received request body for both machines.

```
15:33:26 Machine A CEID 13: processRecipeSelected
15:33:26 [MACHINE A HTTP POST]
15:33:26 Payload: {"machine":"MachineA","timestamp":"2026-08-26T15:33:26+07:00","ceid":13,"event":"processRecipeSelected"}
15:33:26 Machine A HTTP POST skipped: endpoint empty.
```

Figure 5. Machine A event mapped and accepted by the HTTP endpoint.

```
15:33:34 Machine B CEID1: EVENT1
15:33:34 [MACHINE B HTTP POST]
15:33:34 Payload: {"machine":"MachineB","timestamp":"2026-08-26T15:33:34+07:00","ceid":"CEID1","event":"EVENT1"}
15:33:34 Machine B HTTP POST skipped: endpoint empty.
```

Figure 6. Machine B EVENT1 mapped and accepted by the HTTP endpoint.

These results verify the implemented path from equipment event generation through middleware acknowledgement, JSON mapping, and HTTP transmission.

8. Setup and Deployment Instructions

8.1 Prerequisites

- Windows development environment.
- Visual Studio with support for the .NET Framework version used by the solution.
- Network access to the configured HTTP endpoint for telemetry testing.

8.2 Project Layout

```
SECSGEM-Assessment/  
├─ MachineSimulatorA/  
│   ├── Program.cs  
│   └─ MachineSimulatorA.csproj  
├─ MachineSimulatorB/  
│   ├── Program.cs  
│   └─ MachineSimulatorB.csproj  
├─ Middleware/  
│   ├── Program.cs  
│   └─ Middleware.csproj  
└─ SECSGEM-Assessment.sln
```

8.3 Configuration

The simulator ports are defined in source code as 5001 for Machine A and 5002 for Machine B. The middleware uses the same addresses. Before testing HTTP delivery, configure `telemetryEndpoint` locally in `Middleware/Program.cs`. The repository version should keep this value empty so the endpoint is not exposed in source control.

```
static string telemetryEndpoint = "";
```

8.4 Run Procedure

1. Open the solution in Visual Studio and build the three projects.
2. Start `MachineSimulatorA`. It should listen on `127.0.0.1:5001`.
3. Start `MachineSimulatorB`. It should listen on `127.0.0.1:5002`.
4. Start `Middleware`. It connects to both simulators in parallel and runs each setup sequence.
5. Observe Machine A PPSELECT followed by S6F11 / S6F12 and Machine B EVENT1 followed by S6F11 / S6F12.
6. When `telemetryEndpoint` is configured, confirm that the HTTP endpoint accepts both JSON payloads.

8.5 Expected Runtime Behaviour

Once setup is complete, the middleware remains connected and listens for unsolicited equipment messages. If either connection is lost, that worker retries independently after the reconnect delay.

9. Assumptions, Limitations, and Implementation Notes

Area	Note
Transport	Messages use readable SML-like text over TCP. This is not a full binary HSMS implementation.
Framing	<END> is a custom delimiter used by the prototype to frame one TCP message.
Machine A	Numeric SVID/DVID/RPTID/CEID values are based on the supplied Machine A communication scope.
Machine B	RPTID1/RPTID2, SVID1-SVID3, CEID1/CEID2, EVENT1/EVENT2 are symbolic simulation identifiers.
Telemetry	The demonstration maps selected event metadata; it is not intended as a complete equipment data model.
Concurrency	A separate thread is used for each machine connection. The design is adequate for this prototype but is not a production host architecture.
Protocol validation	The implementation validates message names and basic flow but does not implement complete SECS-II type parsing, timers, system bytes, or HSMS session state.
Security	The HTTP endpoint is intentionally not stored in the repository. Production deployments should use protected configuration and appropriate authentication.
Robustness	Production use would require broader error handling, protocol timers, structured logging, configuration management, testing, and monitoring.

9.1 Scope Rationale

The prototype prioritizes the assessment requirements: two machine simulators, a middleware layer, recognizable SECS/GEM communication behaviour, event handling, telemetry mapping, HTTP transmission, and clear documentation. The simplified transport keeps the implementation understandable while preserving the communication sequence being demonstrated.

10. Conclusion

The completed solution demonstrates two independently connected machine simulators communicating with a shared middleware application. Machine A implements the supplied equipment-oriented setup and CEID 13 event path, while Machine B demonstrates a generic report/event model using symbolic identifiers. Both paths produce S6F11 events, receive S6F12 acknowledgements, map events into JSON telemetry, and successfully deliver the payloads to the HTTP endpoint during verification.

The solution should be treated as an assessment prototype rather than a production SECS/GEM host. The implemented architecture and documented limitations provide a clear foundation for extending the project to a proper HSMS/SECS-II library, structured configuration, richer telemetry parsing, and production-grade reliability controls.

Appendix A. Representative Source Snippets

A.1 Machine B Symbolic Mapping

```
reportDefinitions.Add("RPTID1", new string[] {"SVID1", "SVID2"});  
reportDefinitions.Add("RPTID2", new string[] {"SVID3"});  
  
eventReportLinks.Add("CEID1", "RPTID1");  
eventReportLinks.Add("CEID2", "RPTID2");
```

A.2 Telemetry Endpoint Safety

```
static string telemetryEndpoint = "";  
  
if (string.IsNullOrEmpty(telemetryEndpoint)) {  
    Log(machineName + " HTTP POST skipped: endpoint empty.");  
    return;  
}
```

The full source code is provided separately in the Git-based source repository required by the assessment.